BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 5

Roll no: 44 PRN No: 12410016 Name of Student: Sahil Tagade

Subject Name : Deep Learning

Problem Statement

Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

1. Data Preparation

1. Load the sequence classification dataset.
2. Preprocess the input sequences.
3. Convert the sequences into numerical/tokenized form.
4. Pad the sequences to a fixed length.
5. Divide the dataset into training and testing sets.

2. RNN Model

1. Create an embedding layer to convert input tokens into dense vectors.
2. Add a Simple RNN layer to process the sequence.
3. Add a Dense output layer with sigmoid activation for binary classification.
4. Compile the model using Adam optimizer and binary cross-entropy loss.
5. Train the model on the training dataset.

3. LSTM Model

1. Use the same preprocessed input data.
2. Add an embedding layer.
3. Add an LSTM layer to capture long-term dependencies.
4. Add a Dense layer with sigmoid activation.
5. Compile and train the model using the same training conditions.

4. GRU Model

1. Use the same preprocessed input data.
2. Add an embedding layer.
3. Add a GRU layer for sequence processing.
4. Add a Dense layer with sigmoid activation.
5. Compile and train the model.

5. Model Evaluation

Evaluate all three models using:

- Accuracy
- Precision
- Recall
- F1-score
- Loss
- Training/validation performance

6. Comparison

Compare RNN, LSTM, and GRU based on their classification performance, training behavior, and ability to learn sequence dependencies.

Code :

```
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU,
Dense, Dropout
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

#   ============================================================
# Configuration
#   ============================================================
NUM_WORDS = 10000
MAX_LEN = 200
EMBED_DIM = 64
UNITS = 64
EPOCHS = 5
BATCH_SIZE = 128
SEED = 42
RESULTS_DIR = "results"
MODELS_DIR = "models"
```

```
os.makedirs(RESULTS_DIR, exist_ok=True)
os.makedirs(MODELS_DIR, exist_ok=True)

np.random.seed(SEED)
tf.random.set_seed(SEED)

# ============================================================
# 1. Load IMDB dataset
# ============================================================
print("Loading IMDB dataset...")
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=NUM_WORDS)

print(f"Training samples: {len(x_train)}")
print(f"Testing samples : {len(x_test)}")

# ============================================================
# 2. Pad sequences
# ============================================================
x_train = pad_sequences(
    x_train,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)
x_test = pad_sequences(
    x_test,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)
print("Training shape:", x_train.shape)
print("Testing shape :", x_test.shape)
# ============================================================
# 3. Model builder
# ============================================================
def build_model(model_type):
model = Sequential([
        Embedding(
            input_dim=NUM_WORDS,
            output_dim=EMBED_DIM,
            input_length=MAX_LEN
        )
    ])
    if model_type == "RNN":
        model.add(SimpleRNN(UNITS))
    elif model_type == "LSTM":
        model.add(LSTM(UNITS))
    elif model_type == "GRU":
        model.add(GRU(UNITS))
    else:
        raise ValueError("model_type must be RNN, LSTM or GRU")
    model.add(Dropout(0.3))
    model.add(Dense(1, activation="sigmoid"))
    model.compile(
```

```
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
# ============================================================
# 4. Train and evaluate all three models
# ============================================================
model_types = ["RNN", "LSTM", "GRU"]
histories = {}
results = []
reports = {}
for model_type in model_types:
print("\n" + "=" * 70)
print(f"Training {model_type}")
print("=" * 70)
tf.keras.backend.clear_session()
np.random.seed(SEED)
tf.random.set_seed(SEED)
model = build_model(model_type)
model.summary()
start_time = time.time()
history = model.fit(
        x_train,
        y_train,
        validation_split=0.10,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        shuffle=True, verbose=1

    )
    training_time = time.time() - start_time
    # Prediction
    probabilities = model.predict(x_test, batch_size=BATCH_SIZE,
verbose=0)
    predictions = (probabilities >= 0.5).astype(int).ravel()
    # Metrics
    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions, zero_division=0)
    recall = recall_score(y_test, predictions, zero_division=0)
    f1 = f1_score(y_test, predictions, zero_division=0)
    results.append({
        "Model": model_type,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1_Score": f1,
        "Training_Time_sec": training_time
    })
    histories[model_type] = history.history
    report = classification_report(
        y_test,
        predictions,
        target_names=["Negative", "Positive"],
        digits=4,
        zero_division=0
```

```
    )
    reports[model_type] = report
    print(f"\n{model_type} Results")
    print(f"Accuracy : {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall   : {recall:.4f}")
    print(f"F1-score : {f1:.4f}")
    print(f"Training time: {training_time:.2f} seconds")
    print("\nClassification Report:")
    print(report)
    # Save model
    model.save(os.path.join(MODELS_DIR, f"{model_type}.keras"))
    # Confusion matrix
    cm = confusion_matrix(y_test, predictions)
    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(cm)
    ax.set_title(f"{model_type} Confusion Matrix")
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("True Label")
    ax.set_xticks([0, 1], ["Negative", "Positive"])
    ax.set_yticks([0, 1], ["Negative", "Positive"])
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center")
    fig.colorbar(im, ax=ax)
    plt.tight_layout()
    plt.savefig(
        os.path.join(RESULTS_DIR, f"confusion_matrix_{model_type}.png"),
        dpi=150
    )
    plt.close(fig)
# ============================================================
# 5. Save numerical results
# ============================================================
results_df = pd.DataFrame(results)
results_df.to_csv(
    os.path.join(RESULTS_DIR, "metrics.csv"),
    index=False
)
# ============================================================
# 6. Save classification reports
# ============================================================
with open(
    os.path.join(RESULTS_DIR, "classification_reports.txt"),
    "w",
    encoding="utf-8"
) as f:
    for model_type, report in reports.items():
        f.write("=" * 70 + "\n")
        f.write(f"{model_type} Classification Report\n")
        f.write("=" * 70 + "\n")
        f.write(report + "\n\n")
# ============================================================
# 7. Model comparison chart
# ============================================================
metrics = ["Accuracy", "Precision", "Recall", "F1_Score"]
```

```
x = np.arange(len(model_types))
width = 0.18
fig, ax = plt.subplots(figsize=(11, 6))
for i, metric in enumerate(metrics):
    values = results_df[metric].values
    ax.bar(x + (i - 1.5) * width, values, width, label=metric)
ax.set_title("RNN vs LSTM vs GRU - Performance Comparison")
ax.set_xlabel("Model")
ax.set_ylabel("Score")
ax.set_xticks(x)
ax.set_xticklabels(model_types)
ax.set_ylim(0, 1.05)
ax.legend()
plt.tight_layout()
plt.savefig(
    os.path.join(RESULTS_DIR, "model_comparison.png"),
    dpi=150
)
plt.close(fig)

# ============================================================
# 8. Training curves
# ============================================================
fig, ax = plt.subplots(figsize=(11, 6))
for model_type in model_types:
    ax.plot(
        histories[model_type]["accuracy"],
        label=f"{model_type} Train"
    )
    ax.plot(
        histories[model_type]["val_accuracy"],
        linestyle="--",
        label=f"{model_type} Validation"
    )

ax.set_title("Training and Validation Accuracy")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.legend()
plt.tight_layout()
plt.savefig(
    os.path.join(RESULTS_DIR, "training_curves.png"),
    dpi=150
)
plt.close(fig)

# ============================================================
# 9. Print final comparison
# ============================================================
print("\n" + "=" * 80)
print("FINAL MODEL COMPARISON")
print("=" * 80)
print(results_df.to_string(index=False))
best_model = results_df.loc[
    results_df["F1_Score"].idxmax(),
    "Model"
```

```
]
print(f"\nBest model according to F1-score: {best_model}")
print("\nFiles generated in the results/ folder:")
for filename in sorted(os.listdir(RESULTS_DIR)):
    print(" -", filename)
print("\nTrained models generated in the models/ folder:")
for filename in sorted(os.listdir(MODELS_DIR)):
    print(" -", filename)
print("\nAssignment implementation completed successfully.")
```

Output:

1] RNN

```
PS C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\RNN_LSTM_GRU_Sequence_Classification> python compare_rnn_lstm_gru.py
================================================================
Training RNN
================================================================
WARNING:tensorflow:From C:\Users\ThinkPad\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\backend\common\global_state.p
y:82: The name tf.reset_default_graph is deprecated. Please use tf.compat.v1.reset_default_graph instead.

C:\Users\ThinkPad\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:100: UserWarning: Argument `
input_length` is deprecated. Just remove it.
  warnings.warn(
2026-09-05 23:42:00.019930: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU i
nstructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compile
r flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
```

```
176/176 ━━━━━━━━━━━━━━━━━━━━ 26s 145ms/step - accuracy: 0.5958 - loss: 0.6620 - val_accuracy: 0.5008 - val_loss: 0.7121
Epoch 3/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 38s 125ms/step - accuracy: 0.6668 - loss: 0.5901 - val_accuracy: 0.4856 - val_loss: 0.7307
Epoch 4/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 22s 124ms/step - accuracy: 0.7206 - loss: 0.4855 - val_accuracy: 0.4912 - val_loss: 0.7964
Epoch 5/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 22s 126ms/step - accuracy: 0.7500 - loss: 0.4287 - val_accuracy: 0.4972 - val_loss: 0.8809

RNN Results
Accuracy : 0.5103
Precision: 0.5075
Recall   : 0.7014
F1-score : 0.5889
Training time: 137.95 seconds

Classification Report:
              precision    recall  f1-score   support

    Negative     0.5167    0.3193    0.3947     12500
    Positive     0.5075    0.7014    0.5889     12500

    accuracy                         0.5103     25000
   macro avg     0.5121    0.5103    0.4918     25000
weighted avg     0.5121    0.5103    0.4918     25000
```

2] LSTM

```
PS C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\RNN_LSTM_GRU_Sequence_Classification> python compare_rnn_lstm_gru.py

======================================================================
Training LSTM
======================================================================
C:\Users\ThinkPad\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:100: UserWarning: Argument `
input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 73s 362ms/step - accuracy: 0.5261 - loss: 0.6916 - val_accuracy: 0.5576 - val_loss: 0.6868
Epoch 2/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 63s 355ms/step - accuracy: 0.5940 - loss: 0.6619 - val_accuracy: 0.6032 - val_loss: 0.7860
Epoch 3/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 99s 451ms/step - accuracy: 0.5666 - loss: 0.6750 - val_accuracy: 0.6220 - val_loss: 0.6364
Epoch 4/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 85s 466ms/step - accuracy: 0.7274 - loss: 0.5646 - val_accuracy: 0.6760 - val_loss: 0.6435
```

```
Epoch 1/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 73s 362ms/step - accuracy: 0.5261 - loss: 0.6916 - val_accuracy: 0.5576 - val_loss: 0.6868
Epoch 2/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 63s 355ms/step - accuracy: 0.5940 - loss: 0.6619 - val_accuracy: 0.6032 - val_loss: 0.7860
Epoch 3/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 99s 451ms/step - accuracy: 0.5666 - loss: 0.6750 - val_accuracy: 0.6220 - val_loss: 0.6364
Epoch 4/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 85s 466ms/step - accuracy: 0.7274 - loss: 0.5646 - val_accuracy: 0.6760 - val_loss: 0.6435
Epoch 5/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 82s 468ms/step - accuracy: 0.6700 - loss: 0.6102 - val_accuracy: 0.6228 - val_loss: 0.8772

LSTM Results
Accuracy : 0.6216
Precision: 0.5773
Recall   : 0.9087
F1-score : 0.7060
Training time: 402.42 seconds

Classification Report:
              precision    recall  f1-score   support

    Negative     0.7856    0.3346    0.4693     12500
    Positive     0.5773    0.9087    0.7060     12500

    accuracy                         0.6216     25000
   macro avg     0.6815    0.6216    0.5877     25000
weighted avg     0.6815    0.6216    0.5877     25000
```

3] GRU

```
==================================================================
Training GRU
==================================================================
C:\Users\ThinkPad\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:100: UserWarning: Argument `
input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
gru (GRU)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 81s 391ms/step - accuracy: 0.5142 - loss: 0.6926 - val_accuracy: 0.5256 - val_loss: 0.6909
Epoch 2/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 94s 459ms/step - accuracy: 0.5703 - loss: 0.6834 - val_accuracy: 0.5376 - val_loss: 0.6834
Epoch 3/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 78s 435ms/step - accuracy: 0.6488 - loss: 0.6460 - val_accuracy: 0.7612 - val_loss: 0.5541
Epoch 4/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 80s 451ms/step - accuracy: 0.5916 - loss: 0.6818 - val_accuracy: 0.5516 - val_loss: 0.6863
```

```
Epoch 1/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 81s 391ms/step - accuracy: 0.5142 - loss: 0.6926 - val_accuracy: 0.5256 - val_loss: 0.6909
Epoch 2/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 94s 459ms/step - accuracy: 0.5703 - loss: 0.6834 - val_accuracy: 0.5376 - val_loss: 0.6834
Epoch 3/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 78s 435ms/step - accuracy: 0.6488 - loss: 0.6460 - val_accuracy: 0.7612 - val_loss: 0.5541
Epoch 4/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 80s 451ms/step - accuracy: 0.5916 - loss: 0.6818 - val_accuracy: 0.5516 - val_loss: 0.6863
Epoch 5/5
176/176 ━━━━━━━━━━━━━━━━━━━━ 82s 449ms/step - accuracy: 0.5790 - loss: 0.6680 - val_accuracy: 0.5768 - val_loss: 0.6849

GRU Results
Accuracy : 0.5815
Precision: 0.5547
Recall   : 0.8263
F1-score : 0.6638
Training time: 417.54 seconds

Classification Report:
              precision    recall  f1-score   support

    Negative     0.6597    0.3367    0.4459     12500
    Positive     0.5547    0.8263    0.6638     12500

    accuracy                         0.5815     25000
   macro avg     0.6072    0.5815    0.5548     25000
weighted avg     0.6072    0.5815    0.5548     25000
```

```
================================================================================
FINAL MODEL COMPARISON
================================================================================
Model  Accuracy  Precision  Recall  F1_Score  Training_Time_sec
  RNN   0.51032   0.507467 0.70136  0.588864         137.953055
 LSTM   0.62164   0.577273 0.90872  0.706032         402.422008
  GRU   0.58152   0.554726 0.82632  0.663817         417.542166

Best model according to F1-score: LSTM
```

Analysis

The three models were implemented using the same sequence classification dataset and evaluation procedure. RNN is the simplest architecture but may have difficulty learning long-term dependencies because of the vanishing-gradient problem.

LSTM uses memory cells and gating mechanisms to retain important information for longer sequences. Therefore, it is generally more effective when long-term dependencies are important.

GRU is a simplified gated architecture with fewer gates than LSTM. It can provide performance comparable to LSTM while generally requiring fewer parameters and less computation.

The final model should be selected based on the obtained accuracy, precision, recall, F1-score, loss, and training performance shown in the experimental results.